

Projet Génie Logiciel Partie 2

Guillaume Ponsdesserre

Avril 2025

Table des matières

1	Introduction	2
2	Petites Modifications	2
2.1	Renommer une classe, une méthode, une variable	2
2.2	Créer des variables pour supprimer des nombres magiques	2
2.3	Supprimer du code mort	2
2.4	Supprimer du code déprécié	2
2.5	Supprimer du code commenté	2
2.6	Supprimer des imports inutiles	3
2.7	Ajout de Javadoc	3
2.8	Réorganiser une classe pour que le code soit bien structuré	3
3	Modifications Moyennes	3
3.1	Réduire la complexité cyclomatique	3
3.2	Lever une exception plutôt que retourner un code d'erreur	3
3.3	Supprimer la duplication de code	3
3.4	Ajouter un test pertinent	4
3.5	Corriger un test rouge ou orange	4
4	Grandes Modifications	4
4.1	Supprimer les cycles de dépendance entre packages	4

1 Introduction

Comme pour la première partie, je me suis intéressé au projet Webmagic. Je n'ai pas pu aborder toutes les modifications, soit parce qu'elles n'étaient pas présentes dans ce projet, soit parce qu'elles n'étaient pas utiles. Par exemple, je pensais pouvoir réduire le nombre de paramètres de certaines méthodes, cependant, celles-ci étaient déjà bien optimisées et les modifier aurait compliqué le code.

2 Petites Modifications

2.1 Renommer une classe, une méthode, une variable

J'ai renommé la classe `ProcessorBenchmark` en `ProcessorBenchmarkTest` ainsi que la méthode `test` en `testModelPageProcessorPerformance` afin de respecter les conventions de nommage des classes et des méthodes. Cela permet d'identifier qu'il s'agit d'une classe de test et facilite la compréhension du code.

[Lien vers le commit de renommage de la classe](#)

[Lien vers le commit de renommage de la méthode](#)

2.2 Créer des variables pour supprimer des nombres magiques

J'ai créé des constantes dans la classe `ScriptConsole`, pour identifier les différents codes de statut. Cela évite l'utilisation de nombres magiques et améliore la lisibilité.

[Lien vers le commit](#)

2.3 Supprimer du code mort

J'ai supprimé des getters et setters de la classe `ExtractRule`, qui n'étaient jamais utilisés.

[Lien vers le commit](#)

2.4 Supprimer du code déprécié

J'ai supprimé la méthode `scheduler` de la classe `Spider`, qui était dépréciée mais utilisée dans de nombreuses classes, puis j'ai modifié les différentes classes concernées. Cela permet de garder le code propre et d'éviter les confusions.

[Lien vers le commit](#)

2.5 Supprimer du code commenté

J'ai supprimé du code commenté dans la méthode `process` de la classe `AmazonPageProcessor`, ce qui portait à confusion.

[Lien vers le commit](#)

2.6 Supprimer des imports inutiles

J'ai supprimé des imports inutilisés dans la classe `CssSelectorTest`, pour un code plus propre.

[Lien vers le commit](#)

2.7 Ajout de Javadoc

J'ai ajouté de la documentation dans la classe `ModelPageProcessor`, pour une meilleure compréhension des différentes méthodes.

[Lien vers le commit](#)

2.8 Réorganiser une classe pour que le code soit bien structuré

J'ai réorganisé la classe `Spider` pour placer les méthodes publiques en début de classe, suivies des méthodes protégées, puis privées. Cela améliore la lisibilité globale.

[Lien vers le commit](#)

3 Modifications Moyennes

3.1 Réduire la complexité cyclomatique

J'ai réduit la complexité de la méthode `equals` dans `Site`, de 52 à 20, en regroupant toutes les conditions dans une longue expression qui retourne `true` si elles sont toutes vérifiées, et `false` sinon. Cela simplifie aussi la compréhension des comparaisons.

[Lien vers le commit](#)

3.2 Lever une exception plutôt que retourner un code d'erreur

La méthode `getThreadAlive` renvoyait 0 si l'attribut était nul. Je l'ai remplacé par une exception `IllegalStateException`. Cela permet de signaler une erreur d'état plutôt que de retourner une valeur par défaut qui pourrait être interprétée comme valide.

[Lien vers le commit](#)

3.3 Supprimer la duplication de code

J'ai créé la méthode `printResultItems` dans la classe mère `FilePersistentBase`. J'ai ensuite remplacé les parties de code dupliquées des classes filles `OneFilePipeline` et `FilePipeline` par des appels à cette nouvelle méthode. Cela permet de centraliser la logique d'impression des résultats et de réduire la duplication de code. De plus cette méthode est appelée dans deux méthodes `process`, elle pourrait

donc être utile dans d'autres classes qui ont la méthode `process`.

[Lien vers le commit](#)

3.4 Ajouter un test pertinent

J'ai ajouté des tests pour la méthode `isDuplicate` dans `DuplicateStorageRemover` car celle-ci n'avait aucun test, ils sont à exécuter en séquence.

[Lien vers le commit](#)

3.5 Corriger un test rouge ou orange

J'ai corrigé la méthode `process`, dont le comportement était incorrect, ainsi que son test `test` qui n'était pas à jour. Lors de l'exécution globale, le test était vert, mais rouge lorsqu'exécuté individuellement.

[Lien vers le commit](#)

4 Grandes Modifications

4.1 Supprimer les cycles de dépendance entre packages

J'ai supprimé les cycles de dépendance impliquant le package `us/codecraft/webmagic/model`, en plaçant des classes du package `model` dans un dossier commun afin que les dépendances se rejoignent dans ce dossier et donc briser le cycle. Cela permet un code plus modulaire, maintenable, testable et conforme à une architecture claire.

[Lien vers le commit pour les packages formatter et annotation](#)

[Lien vers le commit pour les packages model, pipeline et webmagic](#)

[Lien vers le commit pour les packages model/common, utils et selector](#)